

Knots as Cost–Accounted Rholang Processes: A Dowker–Thistlethwaite–Driven Encoding and Weak–Bisimulation Invariance

L. G. Meredith
F1R3FLY.io

July 27, 2026

Abstract

We revisit a 2005 encoding of knot diagrams as concurrent processes and rebuild it in core rholang (the reflective higher-order ρ -calculus with a built-in name-restriction operator) equipped with cost accounting. A crossing becomes a *gate*: the over-strand passes a signal straight through and produces a private *latch* token; the under-strand is a *join* on its incoming signal and that latch. Chirality is nothing more than which strand produces the latch and which joins on it. The gate is a *recursive definition* that reinstates itself at the latch join: a knot is a standing signalling network, not a one-shot circuit, and a tangle must survive being used in order to be composable. Every receipt stays linear, and the latch is private to one instantiation, so over-enables-under is a matching rather than a count. Driving the construction from a Dowker–Thistlethwaite (DT) code makes every arc a single-producer / single-consumer channel, so the arcs *are* the wires and the whole knot is a flat parallel composition of gates plus injected “current.” Under cost accounting the old *ad hoc* demand for “enough current” becomes a liveness condition on a token ledger, and the current becomes a conserved, metered, hence internal, quantity: the network is a live, bounded, conservative marked graph whose orbit has period $2n$. We sketch a proof that the encoding, taken as a monoidal functor from tangles to processes modulo weak bisimulation, is invariant under the Reidemeister moves, and observe that the *signed* cost of one period is exactly twice the writhe.

1 Introduction

The original intuition [1] was to read a knot diagram as a signalling network. Each crossing is a small gate with two through-paths, one for the over-strand and one for the under-strand. A signal on the over-strand passes straight through; a signal on the under-strand must wait for a *latch* to open, and the latch is opened by the over-strand’s own signal as it passes. Arcs of the diagram carry signals from one gate port to another. The encoding was shown to send the Reidemeister moves to weak bisimulations, making weak-bisimulation class a diagram invariant.

Three ingredients were left implicit. First, because a strand that is over at one crossing may be under at another, the latch dependencies chain, and a single circulating signal can deadlock at an under-strand whose latch has not yet been produced. The original design simply *posited* “enough current” to keep the network live. Second, the gate was written as a bare parallel composition of two linear receives, and such a gate evaporates after one visitation: the network models a single passage through the diagram rather than the diagram itself, and a tangle built from one-shot gates cannot be composed with itself or reused. A knot is a standing network, so a gate must reinstate

itself. Third, the encoding was never driven from a machine-readable knot representation, so it was never mechanically instantiated.

This note repairs all three points. We work in core rholang as implemented by the `f1r3node-rust` interpreter [2], using the built-in `new` (whose several desugarings we do not revisit), recursive definitions written with `def` (rholang’s `contract`), the join receipt written with `&`, and a cost semantics that debits a token from a located stack on each communication. “Enough current” becomes a liveness and conservation invariant of that ledger, and the circulating current becomes internal traffic that weak bisimulation is entitled to erase. We generate the process directly from a DT code, and we recast the invariance result functorially over tangles so that the equivalence being preserved is genuinely observable at the tangle boundary.

2 Core rholang with cost accounting

We use the fragment of rholang generated by

$$P, Q ::= \mathbf{0} \mid x!(Q) \mid \sum_i \text{for}(\vec{y}_i \leftarrow \vec{z}_i)\{P_i\} \mid P \mid Q \mid \text{new } x \text{ in } \{P\} \mid *x \mid \text{def } X(\vec{y}) = P \mid X(\vec{z}),$$

with quoting $@P$ and dereferencing $*x$ realising reflection. A *join* receipt `for`($y_1 \leftarrow z_1 \ \& \ \dots \ \& \ y_k \leftarrow z_k$){ P } fires only when a message is simultaneously available on every z_j , consuming one from each; the general receipt is a `;`-separated list of such `&`-joined groups. Every receipt we use is *linear*: it is consumed by its own firing. We rely only on single-channel receives and two-channel joins.

Recursive definitions. `def` $X(\vec{y}) = P$ binds a name X to a parameterised body in which X may itself occur, and $X(\vec{z})$ instantiates it. We write `def` for what rholang spells `contract`. Two points about its status, both of which matter downstream.

Remark 1 (`def` is derived, and its unfolding is equational). First, `def` is not a new term former. In a reflective calculus recursion comes from reflection: quote the body, pass the quotation to itself, and drop it at each unfolding, so `def` $X(\vec{y}) = P$ abbreviates a term over the formers already listed. This is worth insisting on, because the structural connectives of a logic generated from a theory are read off that theory’s constructors; admitting `def` as primitive would put a corresponding connective into the vocabulary of any generated classifier (Section 10), and we do not want the encoding to enlarge the theory it is encoded in.

Second, unfolding is an *equation*, not a rewrite: $X(\vec{z})$ and $P[\vec{z}/\vec{y}]$ are congruent, exactly as replication unfolds by $!P \equiv P \mid !P$. Unfolding is therefore a congruence step and not a communication, so the cost semantics never charges for it. An implementation may differ, and one should watch for it: rholang’s `contract` is itself a persistent receive on the definition channel, so in `f1r3node-rust` each reinstatement is a real rendezvous. It draws on the stack located at the definition channel rather than at any crossing, so the signed ledger of Remark 5 is untouched; only the unsigned count differs, reading $3n$ per period as implemented against $2n$ in the calculus.

Cost. Fix a commutative monoid of costs $(\mathcal{C}, +, 0)$ with a phlogiston supply. Nothing forbids taking a group, and we do: $\mathcal{C} = \mathbb{Z}$, so that debits carry a sign. The cost semantics attaches to each communication rule a debit: a rendezvous on a channel n consumes one token from the located stack at n (equivalently, one unit of phlogiston) and charges the ledger by the sign ε of the stack’s location. For the gates below, the located stack of crossing i carries the crossing’s sign $\varepsilon_i \in \{+1, -1\}$.

Two quantities are thereby carried by one ledger, and it is essential to keep them apart.

- $N(P)$, the *number* of debits: one unit of phlogiston per rendezvous. This is what solvency is about. A term is *solvent* for a given endowment if no maximal reduction sequence blocks for want of a token.
- $c(P)$, the *signed sum* of debits. This is what the invariant of Section 7 is about.

Because the gates persist, runs are infinite and neither quantity is well defined per run to a normal form; both are counted per *period* (Lemma 1), and every period incurs the same N and the same c up to reordering, so both are well defined.

Remark 2 (Sign is not chirality). The sign ε_i is the usual sign of an oriented crossing — rotate the under-strand’s direction counterclockwise onto the over-strand’s and read off $+1$ or -1 — and it is *not* the same datum as chirality. Chirality says which strand goes over; the sign says that *together with* the two strand directions. The DT traversal orients the curve, so the diagram determines ε_i ; but it is a second bit, independent of which half-gate produces the latch, and the gate carries it as the sign of its located stack rather than in its communication structure. Section 9 needs neither bit, since the hole reading of the gate has no over/under at all.

3 The crossing gate

A crossing has four ports: over-in oi , over-out oo , under-in ui , under-out uo , and a private latch l . The gate is

```
def Gate(oi, oo, ui, uo) = {
  new l in {
    for (s <- oi)      { oo!(*s) | l!(Nil) }           // over: pass, latch
    | for (s <- ui & _ <- l) { uo!(*s) | Gate(oi,oo,ui,uo) } // under: JOIN, respawn
  }
}
```

The over-strand is the unique producer on l ; the under-strand is the unique joiner on l . Nothing else distinguishes the two strands, so a crossing and its mirror differ exactly by swapping producer and joiner. Because the join is atomic, there is no intermediate state in which the under-signal has been consumed while the forward is still stalled: the gate either commits both halves or does nothing, which removes the only awkward interleaving of the nested-*receive* formulation.

Why the gate is recursive, and where it reinstates. Both receipts are linear, so both are spent by their own firing and the body is a one-shot cell. What makes the crossing standing apparatus is that the cell reinstates itself, and the placement of that reinstatement is not free. Putting it in both continuations would release two copies per crossing event and the network would grow without bound; putting it in the over-half alone would arm a fresh over-*receive* while the previous under-half is still outstanding, and the latch of the old copy would be left to be claimed by the new one.

There is exactly one point in the gate at which both strands’ state meets, and it is the latch join. The under-half can fire only if the over-half has already produced the latch, so the firing of the under-join is precisely the witness that one complete crossing event — one over-passage and the under-passage it enabled — has occurred. Reinstating there releases exactly one copy per event, costs no additional channel and no additional rendezvous, and needs no guard: between the over-half firing and the under-half completing there is simply no receiver on oi , so a second over-signal waits because there is nothing to receive it, not because anything tested a condition.

Remark 3 (Matched latch versus counting latch). Because l is created afresh by each instantiation, the latch a given over-passage produces is consumed by the under-passage of the same copy. Over-enables-under is therefore a *matching*. The alternative — a single latch channel shared across all passages, as one gets by making the two receipts persistent instead — is a *counting* latch: k over-passages deposit k tokens and any of them may enable any under-passage, so the pairing is lost and only the count survives. The two are not weakly bisimilar in general, and the difference is observable at an open boundary as soon as more than one signal is in flight on a strand: with a counting latch the over-strand may run ahead of the under-strand, with a matched latch it may not. They agree exactly at the all-arcs current of Definition 1, one token per arc, which is why the distinction does not disturb Theorem 1. We take the matched latch: it is the reading in which the causal skeleton of the diagram — which passage enabled which — is legible in the term, and that skeleton is what Section 10 proposes to keep as the invariant.

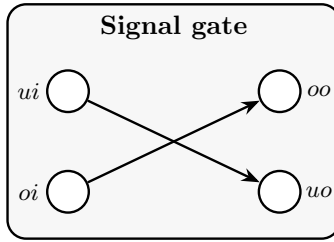


Figure 1: The signal gate: four ports—over-in oi and under-in ui on the left, over-out oo and under-out uo on the right—with the two strands crossing, the over-strand $oi \rightarrow oo$ and the under-strand $ui \rightarrow uo$. In the encoding the over-strand forwards its signal and produces the latch, and the under-strand fires only on the join ($\&$) of its incoming signal with that latch; chirality is which strand plays which.

4 From DT codes to processes

A DT traversal of an n -crossing diagram visits $2n$ passes, numbered $1, \dots, 2n$; each crossing is visited twice, receiving one odd and one even label. The DT code lists, for each odd label $1, 3, \dots, 2n - 1$ in order, the signed even label sharing that crossing. We use the sign convention: $d_i > 0$ iff the odd-labelled pass is the over-strand (so an alternating diagram has all-positive code).

Let arc a_k be the segment from pass k to pass $(k \bmod 2n) + 1$. Then pass p is *entered* by a_{p-1} (with $a_0 \equiv a_{2n}$) and *exited* by a_p . Two facts make the arcs serve as bare channels:

Proposition 1. *In the encoding of a knot diagram every arc a_k occurs exactly once as the output port of a gate and exactly once as the input port of a gate.*

Proof. Each pass contributes exactly one exit (a send on a_p) and one entry (a receive on a_{p-1}). The $2n$ passes are in bijection with the $2n$ arcs on each side, and a knot diagram is a single closed curve, so the exit/entry incidences are a perfect matching on arcs. $\square$

Hence there are no forwarder processes: wiring is name-sharing, and each arc is one metered communication. A short rholang contract `DTtoKnot` and an even shorter Python transliteration emit the term; both accompany this note.

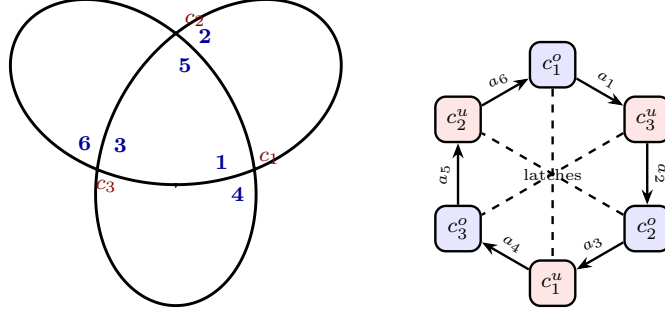


Figure 2: Left: the alternating trefoil, DT code $[4, 6, 2]$, with the six traversal passes numbered and crossings c_1, c_2, c_3 carrying passes $\{1, 4\}, \{2, 5\}, \{3, 6\}$. Right: its encoding. The six half-gates c_i^o, c_i^u sit on a hexagon; the arcs $a_1, \dots, a_6$ (solid) trace the single traversal cycle of the knot (Proposition 1), and the three latch dependencies (dashed) are its diameters.

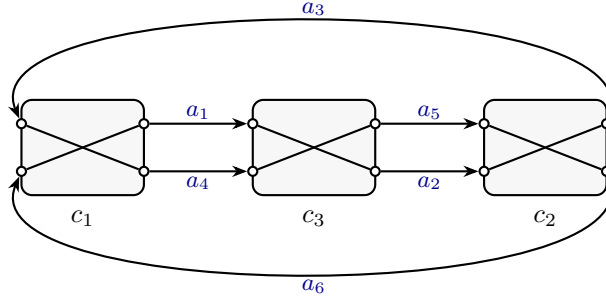


Figure 3: The same trefoil assembled from three signal gates (Figure 1). Each box is one crossing c_i , its over- and under-strands crossing inside; the labelled wires $a_1, \dots, a_6$ are the arcs. All the knotting lives *inside* the gates—the inter-gate wiring is planar, with the two arcs a_3, a_6 closing the traversal into a single loop. Following the closed curve visits each crossing once as over and once as under, recovering the alternating trefoil of Figure 2.

Definition 1 (Encoding). Given a DT code $d = (d_1, \dots, d_n)$, let $\llbracket d \rrbracket$ be the term

$$\text{new } a_1, \dots, a_{2n} \text{ in } \{ (\parallel_i G_i) \mid I \},$$

where G_i is the gate of crossing i with its ports computed as above and $I = \parallel_{k=1}^{2n} a_k! (\mathbf{0})$ is the current.

5 Current, liveness, and cost

With recursive gates the encoding does not run down; it runs. The right statement is therefore not that a run terminates having spent a definite amount, but that the current is conserved and the orbit closes.

Lemma 1 (Liveness, conservation, periodicity). *For the all-arcs current I of Definition 1, $\llbracket d \rrbracket$ is solvent and deadlock-free; the total number of arc tokens is conserved at $2n$; and the reduction is periodic. Once each of the $2n$ half-gates has fired exactly once — n over-receives and n under-joins, the primary rendezvous of a period — the configuration is again one token per arc with no latch outstanding.*

```

// trefoil 3_1, DT [4,6,2] (3 crossings, 6 arcs)
def Gate(oi, oo, ui, uo) = {
  new l in {
    for (s <- oi)      { oo!(*s) | l!(Nil) }
    | for (s <- ui & _ <- l) { uo!(*s) | Gate(oi,oo,ui,uo) }
  }
}

new a1, a2, a3, a4, a5, a6 in {
  Gate(a6, a1, a3, a4)      // crossing 1: over a6->a1, under a3->a4
| Gate(a2, a3, a5, a6)      // crossing 2: over a2->a3, under a5->a6
| Gate(a4, a5, a1, a2)      // crossing 3: over a4->a5, under a1->a2
|
  a1!(Nil) | a2!(Nil) | a3!(Nil) | a4!(Nil) | a5!(Nil) | a6!(Nil) // current
}

```

Figure 4: The trefoil, generated from its DT code. With the gate factored out as a definition, a diagram is just its wiring: three instantiations and the current. Each arc a_k has one producer and one consumer, per Proposition 1, and carries one token per period.

Proof sketch. Read the term as a marked graph. There are three kinds of place: the $2n$ arcs, the n latches, and — because the gate is a recursive definition rather than a pair of standing receives — the n *arming* places g_i , one per crossing, holding a token exactly when an instantiation of G_i is present with its over-half unspent. The transitions are the $2n$ half-gates. The initial marking puts one token on each arc and one on each g_i ; unfolding G_i is a congruence step (Remark 1) and so is not a transition. By Proposition 1 each arc is the output place of exactly one transition and the input place of exactly one.

The two half-gates then read

$$\text{over}_i: g_i + a_{oi} \longrightarrow l_i + a_{oo}, \quad \text{under}_i: l_i + a_{ui} \longrightarrow g_i + a_{uo},$$

the g_i on the right of under_i being the reinstatement.

Liveness. At the initial marking every over_i has its arming token and a token on its over-in arc, hence is enabled; firing it deposits a latch and a token on the over-out arc. Each under_i then has its under-in token and its latch and is enabled; firing it restores g_i . No transition is permanently starved.

Conservation. Each of the three place classes is conserved: arc tokens outright, at $2n$; arming tokens outright, at n , so exactly one instantiation of each gate is live at any time and the network does not grow; and latches created and destroyed in matched pairs, one pair per crossing per period. The arming invariant is the marked-graph face of Remark 3: it is what forbids a second over-passage before the first under-passage completes.

Periodicity. In a marked graph the marking is determined by the initial marking and the firing vector. Each arc appears once as an input and once as an output, and each g_i and l_i once as each, so the firing vector in which every half-gate fires once leaves the marking unchanged, whatever the interleaving. $\square$

Under the all-arcs policy there is thus no “surplus current” and no residue: the quantity the original design tolerated as slack is, with recursive gates, precisely the conserved circulating charge, and the ledger measures it rather than tolerating it. The *minimal* live endowment — the least

placement of tokens on arcs admitting a deadlock-free orbit — is the genuine optimisation problem hiding here; it coincides with the phlogiston budget of one period and we leave it open.

6 Tangles, boundaries, and the encoding functor

To make “weak bisimulation” meaningful we work with *open tangles*. A tangle T with p inputs and q outputs is a piece of diagram with $p+q$ boundary endpoints; composition glues outputs to inputs, and juxtaposition places tangles side by side. Tangles form a (braided, framed) monoidal category **Tang** whose morphisms modulo ambient isotopy rel boundary are generated by cups, caps, the two braidings, and the identity, subject to the Reidemeister relations [4].

The encoding of a tangle keeps its boundary arcs *free*: the p input arcs and q output arcs are not restricted, so they are observable channels. Let $\mathbf{Proc}_\approx$ be rholang processes-with-boundary modulo weak bisimulation $\approx$, where $\approx$ abstracts the internal (latch and forward) τ -steps but observes communication on free boundary names. Extend Definition 1 to open tangles by supplying local current on internal arcs only.

Definition 2. $\llbracket - \rrbracket : \mathbf{Tang} \rightarrow \mathbf{Proc}_\approx$ sends a tangle to its gate network with free boundary; composition maps to boundary-name identification and juxtaposition to parallel composition.

Lemma 2 (Boundary forwarder). *For every tangle T , $\llbracket T \rrbracket$ is weakly bisimilar to the persistent pure forwarder that copies each input boundary signal to the output boundary endpoint reached by following the strand through T , and does so on every passage. All crossing machinery is internal.*

Proof sketch. By Lemma 1 applied locally, each gate relays an incoming signal to the corresponding outgoing port through a bounded run of τ -steps, consuming its private latch, and — because the receipts persist — is left in a state ready to do so again, its arming token restored. Composing gates along internal arcs composes persistent forwarders; the internal arcs and latches are restricted, hence their traffic is τ . What remains observable is precisely the input $\rightarrow$ output relaying at the free boundary, available at every passage. $\square$

Remark 4 (Reinstatement is what makes the functor a functor). The forwarder of Lemma 2 must be persistent for Definition 2 to define a functor at all. Composition in **Tang** glues boundaries, and the glued tangle is itself a morphism that may be composed again and traversed again; a forwarder good for one passage would make $\llbracket - \rrbracket$ land in single-use processes, where composition is not associative in any useful sense because an intermediate tangle can be spent. One-shot gates would give an encoding of a *traversal* of a diagram; reinstating gates give an encoding of the diagram.

Lemma 3 (Congruence). *$\approx$ is a congruence for parallel composition and name restriction. Hence if $\llbracket T \rrbracket \approx \llbracket T' \rrbracket$ for tangles with equal boundary, then $\llbracket D[T] \rrbracket \approx \llbracket D[T'] \rrbracket$ in every diagram context $D[-]$.*

Proof. Standard for the ρ -calculus: $|$ and **new** preserve weak bisimulation. Boundary identification is a substitution of free names, which $\approx$ also respects. $\square$

7 Weak-bisimulation invariance

Theorem 1 (Reidemeister invariance). *If diagrams D and D' are related by a finite sequence of Reidemeister moves, then $\llbracket D \rrbracket \approx \llbracket D' \rrbracket$. Consequently $\llbracket - \rrbracket$ descends to a map from knot types to weak-bisimulation classes.*

Proof sketch. By Lemma 3 it suffices to verify the three moves as local tangle relations; each relates two tangles with the same boundary.

Type I (twist). The kinked $1 \rightarrow 1$ tangle has a single self-crossing: one strand is both over and under, its two passes consecutive. The over-pass produces the latch that the same strand's under-pass immediately joins on, so the input signal reaches the output through two internal τ -steps. By Lemma 2 the kink and the bare wire are the same $1 \rightarrow 1$ forwarder, hence weakly bisimilar. The cost differs: the kink adds one over-receive and one under-join at a single new crossing of sign ε , so $\Delta N = +2$ and $\Delta c = 2\varepsilon = 2 \Delta w$.

Type II (poke). In the removable configuration one strand A passes over the other strand B at both crossings. A forwards straight through, producing two latches; B 's signal passes two under-joins, each gated by one of A 's latches. The result is two independent forwarders $A: a \rightarrow a'$ and $B: b \rightarrow b'$, identical to the two parallel wires; weak bisimilarity again follows from Lemma 2. Here the two ledgers part company, and this is the point of signing them. The move adds two over-receives and two under-joins, so $\Delta N = +4$; but the two crossings it creates have *opposite* signs, so their debits cancel and $\Delta c = 2(+1) + 2(-1) = 0 = 2 \Delta w$. Both deltas are local to the two crossings.

Type III (slide). Both tangles have three strands and three crossings, and the move preserves, pairwise, which strand of each crossing is the over-strand, as well as the input $\rightarrow$ output connectivity of the three strands. Only the *location* of the crossings changes. Thus the two encodings induce the same boundary forwarder and differ only in the internal τ -schedule, so they are weakly bisimilar. Here both ledgers are unchanged: both sides have the same three crossings with the same three signs, i.e. six primary rendezvous per period, so $\Delta N = \Delta c = 0$.

Weak bisimilarity of the local tangles lifts to $\llbracket D \rrbracket \approx \llbracket D' \rrbracket$ by Lemma 3, and the Reidemeister moves generate ambient isotopy. $\square$

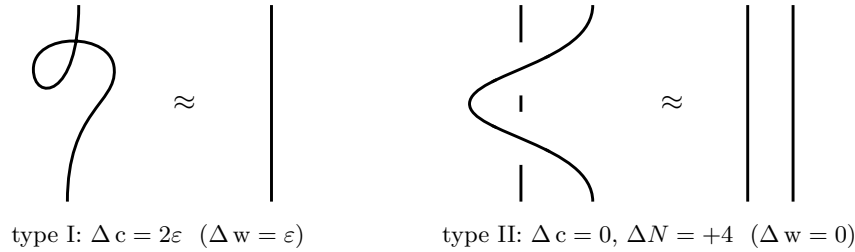


Figure 5: The two crossing-changing moves as local tangle equivalences. Left, type I: the kink and the bare strand are the same $1 \rightarrow 1$ boundary forwarder; the kink adds one over-receive and one under-join at a single crossing of sign ε , so $\Delta c = 2\varepsilon = 2 \Delta w$. Right, type II: the over-passing bigon and the two parallel strands are the same $2 \rightarrow 2$ forwarder; it adds four primary rendezvous, $\Delta N = +4$, but at two crossings of opposite sign, so the signed ledger cancels, $\Delta c = 0$, matching $\Delta w = 0$. This is the case that distinguishes the two ledgers (Remark 6). In both moves the added machinery is internal (τ), so weak bisimulation absorbs it.

Remark 5 (Cost counts the writhe). Summing the three cases: types II and III leave c invariant — cancelling and preserving respectively — while type I changes it by 2ε per twist. Since $w(D) = \sum_i \varepsilon_i$ and every rendezvous of a period is charged at its crossing's sign, with each crossing contributing two primary rendezvous per period (Lemma 1),

$$c(\llbracket D \rrbracket) = 2 \sum_i \varepsilon_i = 2 w(D)$$

identically, and not merely up to a regular-isotopy constant: *the signed cost of one period is exactly twice the writhe*. If an unframed invariant is wanted, one normalises by it, exactly as the Kauffman bracket is normalised by a writhe factor to yield the Jones polynomial. The encoding is thus a regular-isotopy (framed) invariant on the nose, with the framing read directly off the ledger.

Remark 6 (Why the debits must be signed). The unsigned count does *not* do this job, and it is worth saying so plainly because the reading is tempting. By Lemma 1 a period consists of $2n$ primary rendezvous, so $N(\llbracket D \rrbracket) = 2n(D)$ is twice the crossing number. That is not the writhe, and it is not even a regular-isotopy invariant: a type-II move leaves the writhe alone but changes N by $+4$. The two quantities agree only on diagrams whose crossings all have the same sign — for instance on the standard alternating trefoil, where $n = 3$, $w = +3$ and both ledgers read 6, which is exactly the coincidence that makes the unsigned reading look adequate. Signing the debits is what separates n from w , and the sign is available to the ledger for free, since each crossing’s located stack is already a distinguished place at which to record it (Remark 2).

8 The Perko pair

The sharpest classical test for any diagram-level construction is the *Perko pair*: the two reduced 10-crossing diagrams Perko A (10_{161}) and Perko B (formerly 10_{162}), listed as distinct knots from Little (1885) through Rolfsen (1976) until Perko proved them the same knot in 1974 [5, 6]. It is the smallest instance of two reduced diagrams of the same prime knot that a naive tabulation separated, and—this is the point that matters for us—the two diagrams have *different writhes*; the pair is the historical counterexample to Little’s 1900 claim that the writhe of a reduced diagram is a knot invariant [7]. Perko A has DT code

$$[4, 12, -16, 14, -18, 2, 8, -20, -10, -6],$$

from which $\llbracket - \rrbracket$ produces a 10-gate, 20-arc network; every arc is a single producer/single consumer (Proposition 1), and a period is 20 primary rendezvous. The generated term accompanies this note.

The pair probes the encoding from two sides, and it passes both.

Invariance: the theorem must *not* separate them. Perko A and Perko B are the same knot, hence connected by Reidemeister moves, so Theorem 1 gives $\llbracket A \rrbracket \approx \llbracket B \rrbracket$ as boundary forwarders, and after closing the boundary $\llbracket A \rrbracket_{\text{cl}} \approx \llbracket B \rrbracket_{\text{cl}}$ by the congruence of $\approx$ under restriction (Lemma 3). What makes the Perko pair famously hard for humans is that Perko’s connecting sequence is not monotone in crossing number: it must pass through diagrams with *more* than ten crossings. Our proof is indifferent to this. It never tracks crossing number or any complexity measure along the connecting path; it factors the equivalence through the three local relations and lifts by congruence. The crossing-number-increasing detour that defeats a greedy simplifier is invisible to the argument, so the encoding is not fooled the way the tables were.

Framing: the cost *does* separate them, and should. The pair also settles which ledger is the right one. Perko A and Perko B are both reduced 10-crossing diagrams, so $N(\llbracket A \rrbracket) = N(\llbracket B \rrbracket) = 20$: the unsigned count cannot tell them apart, because it is measuring crossing number, which they share. Because the two diagrams have different *writhes*, Remark 5 gives

$$c(\llbracket A \rrbracket) - c(\llbracket B \rrbracket) = 2(w(A) - w(B)) \neq 0.$$

Thus the encoding assigns the pair the *same* weak-bisimulation class (correctly: one knot) while recording *distinct* cost ledgers (correctly: two framings). The unframed invariant is recovered after the writhe normalisation of the remark; the framed, regular-isotopy reading keeps A and B apart exactly where they genuinely differ.

Why this is a confirmation, not a threat. The signed cost of the encoding is twice the writhe—precisely the quantity Little’s 1900 “theorem” wrongly asserted to be a knot invariant, and precisely the quantity the Perko pair refutes. Theorem 1 states the correct status of that quantity: it is a regular-isotopy (framed) invariant, not an ambient-isotopy one. The Perko pair instantiates that very distinction at minimal crossing number. Far from embarrassing the construction, it shows the encoding cleaves the data at the right joint: knot type lands in the weak-bisimulation class, framing lands in the cost, and the two are cleanly separated. The atom of the phenomenon is already the type-I move—a kink on the unknot is the smallest pair of same-knot diagrams with different writhes, contributing the +2 our ledger charges; the Perko pair is the smallest place where the accumulated effect distinguishes two reduced diagrams of a nontrivial prime knot.

9 The Jones polynomial via resolution-contexts

The gate has a second life. Read dynamically it is a latch; read structurally it is a four-port *hole*, and the Kauffman bracket is exactly what one computes by plugging those holes.

Smoothings are contexts. A crossing tangle has four boundary ports. There are two planar ways to reconnect them without a crossing—the two *smoothings*. In process terms each is a pair of wires, i.e. a *context* into which the gate-hole is plugged. Writing the four ports as top-left, top-right (inputs) and bottom-left, bottom-right (outputs), abbreviated TL, TR, BL, BR, the two connectors are

$$\begin{aligned} \mathbf{1} &= \{ \text{TL-BL}, \text{TR-BR} \} \quad (\text{two through-wires}), \\ e &= \{ \text{TL-TR}, \text{BL-BR} \} \quad (\text{cup-cap}). \end{aligned}$$

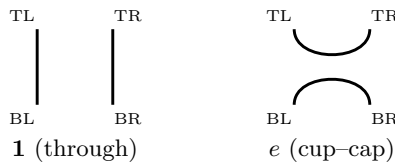


Figure 6: The two resolution-contexts of a gate-hole: the through-connector $\mathbf{1}$ and the cup-cap e — the two diagrams of TL_2 . Kauffman’s relation reads $\text{crossing} \mapsto A\mathbf{1} + A^{-1}e$, and a closed loop of pure wire is worth δ (so $e^2 = \delta e$).

Replacing every gate in $\llbracket D \rrbracket$ by one of its two connectors yields a process of pure wire: a disjoint union of closed loops. A closed loop is a τ -cycle with no free port; its scalar value is $\delta = -A^2 - A^{-2}$.

The bracket, on the process term. We can run the whole computation on the process itself, never leaving the term. By Definition 1 the image of an n -crossing knot is

$$\llbracket K \rrbracket = \text{new } \bar{w} \text{ in } \prod_i \text{Gate}(w_{i1}, w_{i2}, w_{i3}, w_{i4}),$$

so a knot is literally a process *context* with n gate-holes, $K[\text{Gate}, \dots, \text{Gate}]$. Kauffman's relation, applied at one hole, is then a recursion on the term:

$$\langle K[\text{Gate}] \rangle = A \langle K[\mathbf{1}] \rangle + A^{-1} \langle K[e] \rangle,$$

where $K[\mathbf{1}]$ and $K[e]$ are the *same* process with that one gate replaced by a wiring—each a pure name-identification on the term, merging the port pairs one way or the other. Recurring over the remaining holes generates the 2^n leaves from the context algebra alone; the base case $K[]$ (no gates left) is a pure-wire process, a disjoint union of closed τ -cycles, valued $\delta^{\#\text{loops}-1}$. There is no external state sum to import: the bracket is computed by peeling gate-holes off $\llbracket K \rrbracket$ and identifying names.

Two things make this more than bookkeeping. First, the orientation data the bracket needs is not imposed from outside—it is read off the process. The gate already carries the chirality (which strand produces the latch and which joins on it), and that datum is exactly what fixes which of $K[\mathbf{1}]$, $K[e]$ receives A and which receives A^{-1} . A partially resolved knot and a process-with-a-hole are the same object, so the context decomposition and the skein relation are one operation. Second, this recursion is the monoidal-linear functor of Section 6 made constructive: the encoding functor and the Kauffman state sum are a single statement, evaluated by peeling holes.

The state sum. Unfolding the recursion to all n holes gives the familiar closed form. A *state* s assigns a smoothing to each of the n crossings; let $a(s)$ and $b(s)$ count the A - and B -smoothings and $|s|$ the number of resulting loops. Then

$$\langle D \rangle = \sum_s A^{a(s)-b(s)} \delta^{|s|-1}, \quad \delta = -A^2 - A^{-2},$$

where the loop count $|s|$ is read directly off the pure-wire leaf as its number of τ -cycles—the leaves the recursion above enumerates.

Temperley–Lieb, realised in the calculus. The recursion never leaves rholang, and neither does the algebra it lands in. Its leaves are pure-wire processes with free boundary, composed by name-identification, and up to weak bisimulation this fragment *is* the Temperley–Lieb category $\text{TL}(\delta)$: on four ports the two wirings $\mathbf{1}$ and e are its generators, sequential composition is boundary-name identification of processes, and the defining relation

$$e^2 = \delta e, \quad \delta = -A^2 - A^{-2},$$

is not an axiom we impose but a reduction we observe—plugging e into e closes one wire into a τ -cycle, which weak bisimulation collapses to the scalar δ . Closing a knot restricts its boundary names (Section 6), and the Markov trace that turns a resolved state into a number is exactly that closure: each surviving τ -cycle contributes one factor of δ . So the Kauffman bracket is the linear functor

$$\langle - \rangle: \mathbf{Tang} \longrightarrow \mathbb{Z}[A^{\pm 1}]\text{-}\mathbf{Mod}, \quad \text{crossing} \mapsto A \mathbf{1} + A^{-1} e,$$

but it *factors through an algebra we never had to supply*: $\text{TL}(\delta)$ is the planar-wire sub-quotient of the process calculus itself, presented by name-identification and loop-collapse. The abstract multicategory of resolution-contexts and the concrete wire fragment of rholang are the same thing.

Worked example: the trefoil. Carrying out the recursion on the three gate-holes at once, present the trefoil as the closure of the 2-strand braid σ_1^3 . Each σ_1 maps to $A\mathbf{1} + A^{-1}e$ in TL_2 ; cubing (using $e^2 = \delta e$) gives

$$(A\mathbf{1} + A^{-1}e)^3 = A^3\mathbf{1} + (A - A^{-3} + A^{-7})e.$$

The trace closure sends $\mathbf{1} \mapsto \delta$ (two loops) and $e \mapsto 1$ (one loop), so

$$\langle 3_1 \rangle = A^3\delta + (A - A^{-3} + A^{-7}) = -A^5 - A^{-3} + A^{-7}.$$

From bracket to Jones, and the writhe again. The bracket is invariant under the type-II and type-III moves but scales by $-A^{\pm 3}$ under type-I. The normalisation

$$f(D) = (-A^3)^{-w(D)} \langle D \rangle$$

is invariant under all three moves, and the Jones polynomial is $V_K(t) = f(D)|_{A=t^{-1/4}}$. For the trefoil, $w = +3$ gives $f = A^{-4} + A^{-12} - A^{-16}$ and hence

$$V_{3_1}(t) = -t^4 + t^3 + t.$$

The normalisation exponent is the writhe—the very quantity our cost ledger records as $2w$ (Section 8). So the bracket-to-Jones passage, which trades the framed (regular-isotopy) bracket for the ambient-isotopy Jones invariant, is the *same* writhe quotient as the passage from our regular-isotopy encoding to an ambient one. One writhe, two faces: on the state-sum side it is the $(-A^3)^{-w}$ factor; on the process side it is the signed cost of a period. The two halves of this note—weak bisimulation with cost, and the Kauffman state sum—meet exactly at the writhe, and the gate’s two readings keep them apart cleanly: the latch reading carries the dynamics and the current, while the hole reading, having no over/under, computes the bracket with no current at all.

Programs as invariants. The Jones polynomial was a step in the right direction, and the reason is precisely that it is a *dynamical* object: a partition function over resolutions, not a static labelling. But it discards the dynamics at the end, keeping only the polynomial shadow. Computing the bracket on $\llbracket K \rrbracket$ suggests keeping the dynamics instead—taking the invariant to be the process itself, up to weak bisimulation. The signal-processing structure implicit in the knot (over-enables-under, and with it the entire causal skeleton of the diagram) is then legible directly in the invariant rather than integrated away; rholang is its natural home because that dynamics is exactly what the calculus is made of, and the context decomposition fits the Kauffman bracket hand in glove because a partially resolved knot and a process-with-a-hole are the same object. Most importantly, once knot equivalence is cast as a weak-bisimulation statement, it is a beachhead: coinduction, bisimulation-up-to, and the OSLF-generated logics (adequate when the target specification supplies the connectives for it) become available as proof technique *in topology*, importing the methods of concurrency theory into mainstream mathematics through the crossing. This is the direction the companion note pursues.

10 Outlook

A spatial-behavioural classifier. Theorem 1 gives invariance; the converse, distinguishing non-isotopic knots, is the open half, and the companion note is about it. Three things must be said about its status, because an earlier version of this note ran them together.

Closing is very lossy. Closing a tangle into a knot restricts all boundary names, so every step becomes internal. Weak bisimulation abstracts exactly the internal steps, and a closed encoding is a live, periodic τ -network (Lemma 1); as an observable object it is nearly featureless, and the weak-bisimulation class of $\llbracket K \rrbracket_{\text{cl}}$ cannot be expected to carry knot type on its own. The discriminating content has to come from somewhere else: either from the *spatial* structure of the term — the arrangement and nesting of gates and latches — or from keeping the boundary open and comparing tangles by *context-labelled* transitions, where the observing contexts are restricted to the image of the tangle contexts rather than ranging over all rholang contexts. The second route is the one that inherits Theorem 1 directly, since the theorem is proved at the boundary; the first needs its own invariance argument, because gate count and gate arrangement are manifestly not preserved by the Reidemeister moves.

Faithfulness has a name already. Under the context-labelled reading, the question “is the induced equivalence as fine as knot type?” is the question whether $\llbracket - \rrbracket: \mathbf{Tang} \rightarrow \mathbf{Proc}_{\approx}$ *reflects* as well as preserves the context-labelled transition structure — that no branching present in **Tang** is lost in the image. That is the standing condition on maps of graph-structured lambda theories, and stating the conjecture in those terms is what makes it attackable rather than merely hopeful.

Adequacy is not free, and is in tension with checkability. It is tempting to say that OSLF hands us adequacy and only faithfulness remains. It does not. Adequacy — two closed terms satisfy the same formulae iff they are bisimilar — is a property of the *target logic specification*, not of the generating machinery [3]; the Hennessy–Milner argument needs something like a conjunction, to record several observations of one state, and something like a negation, to record the absence of an observation. Members of the OSLF family that generate type systems rather than modal logics carry no adequacy theorem at all. Worse, the property one actually wants from a classifier pulls the other way. A logic whose model-checking terminates is obtained by restricting to a fragment of the theory, or of the connectives, or both — and restricting the theory necessarily restricts the spatial and behavioural connectives, since those are generated from its constructors and rewrites, while the connectives coming from the collection semantics can force non-termination independently and may need restricting on their own account. A logic with terminating model-checking is therefore necessarily not adequate for the whole of a Turing-complete host. Note also that a classifier wants more than adequacy: it wants a computable $K \mapsto \chi_K$ with $\llbracket K' \rrbracket \models \chi_K$ iff K' is K , i.e. characteristic formulae, which are built from precisely the fixpoint or infinitary conjunctions one drops first when chasing decidability.

None of this is fatal, and the reason is that the host need not be Turing complete. **Tang** is a small, finitely presented theory — no replication, no reflection, no way to code its own contexts — and asking for an adequate, finitely checkable logic *there* is a different and far more modest request than asking for one over rholang. Which of the two the companion note should take as its base theory is the first thing it has to decide. With that settled, we expect the spatial-behavioural logic to be a sharper practical classifier than Conway’s notation, which is a naming scheme tuned to rational tangles rather than an intrinsic, isotopy-respecting invariant.

Minimal current. The least live endowment—the smallest placement of tokens on arcs admitting a deadlock-free orbit, and so the true phlogiston budget of a diagram—may itself be an isotopy invariant, and its behaviour under the Reidemeister moves is the natural next computation. Persistence sharpens the question rather than settling it: the all-arcs current is now visibly a conserved circulating charge (Lemma 1), so the problem is to find the minimal conserved charge supporting a live orbit, which is a question about the cycle structure of the latch dependencies rather than about the length of a run.

A Rholang source

The listing below is the accompanying file `knot_encoding.rho` in full: the hand-expanded trefoil (Part 1), the parametric DTtoKnot builder (Part 2), and the Perko A term of Section 8 (Part 3). It targets the `f1r3node-rust` rholang interpreter, where `def` is spelled `contract`; every gate receipt is linear (`<-`) and the standing character of a crossing is carried by the definition alone, per Remark 1; every send and receive carries a payload (the interpreter rejects 0-arity), no receive is guarded, and no arithmetic appears in a match pattern.

```
// =====
// Knots as rholang processes -- F1R3FLY.io working code
// Target: f1r3node-rust rholang-cli
//
// Crossing gate (four ports oi,oo,ui,uo; private latch l):
//   over strand : receive on oi, forward on oo, PRODUCE the latch l
//   under strand: JOIN on (ui, l) with '&', forward on uo, REINSTATE the gate
// Chirality == which strand produces l (over) vs. joins on it (under).
//
// RECURSION, NOT PERSISTENT RECEIPTS. Every receipt below is LINEAR ('<-').
// The gate body is therefore a one-shot cell; what makes a crossing standing
// apparatus is that the cell reinstates itself. The reinstatement sits in
// the under-half's continuation, and that placement is forced: the under-join
// can fire only after the over-half produced the latch, so its firing is the
// unique witness that one COMPLETE crossing event -- one over-passage and the
// under-passage it enabled -- has occurred. Reinstating there releases
// exactly one copy per event (both continuations would double the copies each
// event; the over-half alone would arm a fresh over-receive while the old
// under-half is still outstanding). It needs no guard and no extra channel.
//
// MATCHED LATCH. Because 'new l' runs afresh on each instantiation, the
// latch an over-passage produces is consumed by the under-passage of the SAME
// copy: over-enables-under is a matching, not a count. (Making both receipts
// persistent instead would share one latch channel across all passages, which
// counts but does not pair.) A consequence: between an over-firing and the
// matching under-firing there is no receiver on oi at all, so a second
// over-signal simply waits -- structurally, not by any test.
//
// A DT traversal makes every arc a single-producer / single-consumer
// channel, so the arcs ARE the wires: no forwarder processes.
// "Current" is one message per arc. It is CONSERVED rather than spent: arc
// tokens are neither created nor destroyed, latches are created and destroyed
// in matched pairs, exactly one copy of each gate is live at any time, and
// once every half-gate has fired once the marking is back where it started.
// A period is 2n primary rendezvous (n over-receives + n under-joins).
//
// NB in the calculus, unfolding a definition is an EQUATION and costs
// nothing. In this interpreter 'contract' is a persistent receive on the
// definition channel, so each reinstatement is a real comm. It draws on the
// stack located at the definition channel, not at any crossing, so the signed
// ledger (= 2*writhe) is unaffected; only the unsigned count reads 3n per
// period here against 2n in the calculus.
//
// NB on interpreter limits (rholang/README.md): no 0-arity send/receive
// (we always carry Nil), no guarded receive patterns, match patterns are
// not pre-evaluated (we never put arithmetic in a pattern).
// =====

// -----
// Part 1. The trefoil 3_1, DT code [4,6,2]. Runnable.
//
// passes 1..6, arc a_k = pass k -> pass (k+1 mod 6). pass p: in a_{p-1}, out a_p.
// alternating => odd pass is OVER at every crossing (all-positive DT).
// crossing 1 {odd 1, even 4}: over a6->a1, under a3->a4
// crossing 2 {odd 3, even 6}: over a2->a3, under a5->a6
```

```

// crossing 3 {odd 5, even 2}: over a4->a5, under a1->a2
// -----
new Gate, a1, a2, a3, a4, a5, a6, stdout('rho:io:stdout') in {

  contract Gate(oi, oo, ui, uo) = {
    new l in {
      for (s <- oi) { oo!(*s) | l!(Nil) }
      | for (s <- ui & _ <- l) { uo!(*s) | Gate!(*oi, *oo, *ui, *uo) }
    }
  }

  |
  Gate!(*a6, *a1, *a3, *a4) // crossing 1
  |
  Gate!(*a2, *a3, *a5, *a6) // crossing 2
  |
  Gate!(*a4, *a5, *a1, *a2) // crossing 3
  |
  // current: one token per arc. Conserved; the orbit has period 6.
  a1!(Nil) | a2!(Nil) | a3!(Nil) | a4!(Nil) | a5!(Nil) | a6!(Nil)
  |
  stdout!("trefoil encoding running")
}

// =====
// Part 2. Parametric builder: DT code -> knot process.
//
// DTtoKnot(dt, ret): dt is a list of signed even labels (one per crossing).
// Builds the whole network from a fresh unforgeable token 'knot'; arc k is
// the reflected name @[*knot, k]. Returns *knot on ret so a caller can
// seed / observe the boundary if desired.
//
// Convention: dt_i > 0 => the ODD-labelled pass is the OVER strand.
// (Flip the (over,under) assignment below for the opposite convention.)
// =====
new DTtoKnot, Gate, length in {

  // the crossing gate, once, for all crossings of all knots built here
  contract Gate(oi, oo, ui, uo) = {
    new l in {
      for (s <- oi) { oo!(*s) | l!(Nil) }
      | for (s <- ui & _ <- l) { uo!(*s) | Gate!(*oi, *oo, *ui, *uo) }
    }
  }

  // length of a list
  contract length(xs, ret) = {
    match xs {
      [] => ret!(0)
      [_ ...t] => { new r in { length!(t, *r) | for (@m <- r) { ret!(m + 1) } } }
    }
  }

  contract DTtoKnot(@dt, ret) = {
    new knot, gates, inject, lenCh in {
      length!(dt, *lenCh) |
      for (@len <- lenCh) {
        // N = 2n arcs
        new go in {
          // ---- instantiate one Gate per crossing ----
          // gates(remaining, i, N): i is 1-based crossing index (odd label = 2i-1)
          contract gates(@rem, @i, @nn) = {
            match rem {
              [] => Nil
              [d ...t] => {
                // even label and over/under choice
                new eCh in {
                  ( if (d < 0) { eCh!(0 - d) } else { eCh!(d) } ) |

```

```

    for (@e <- eCh) {
      new pick in {
        // pick!(overPass, underPass)
        ( if (d > 0) { pick!(2 * i - 1, e) } else { pick!(e, 2 * i - 1) } ) |
        for (@over, @under <- pick) {
          new oiCh, uiCh in {
            // in-arc(p) = N if p==1 else p-1 ; out-arc(p) = p
            ( if (over == 1) { oiCh!(nn) } else { oiCh!(over - 1) } ) |
            ( if (under == 1) { uiCh!(nn) } else { uiCh!(under - 1) } ) |
            for (@oi <- oiCh & @ui <- uiCh) {
              Gate!( *@[*knot, oi],      *@[*knot, over],
                    *@[*knot, ui],      *@[*knot, under] )
            }
          }
        }
      }
    } |
    gates!(t, i + 1, nn)
  }
}
} |
// ---- inject current on every arc 1..N ----
contract inject(@k, @nn) = {
  if (k <= nn) { @[*knot, k]!(Nil) | inject!(k + 1, nn) } else { Nil }
} |
gates!(dt, 1, 2 * len) |
inject!(1, 2 * len) |
ret!(*knot)
}
}
} |
}

// Example: build the trefoil and the figure-eight
new r1, r2 in {
  DTtoKnot!([4, 6, 2], *r1) |
  DTtoKnot!([4, 6, 8, 2], *r2) |
  for (_ <- r1) { Nil } |
  for (_ <- r2) { Nil }
}
}

// =====
// Part 3. The Perko pair. Perko A = 10_161, DT [4,12,-16,14,-18,2,8,-20,-10,-6]
// (KnotInfo / Wikipedia). Perko A and Perko B (formerly 10_162) are the SAME
// knot with DIFFERENT writhes -- the classical refutation of Little's
// writhe-invariance claim. Our encoding gives them weakly-bisimilar boundary
// forwarders (same knot) but SIGNED cost ledgers differing by 2*(w_A - w_B).
// The UNSIGNED count does not separate them: both are reduced 10-crossing
// diagrams, so both spend 20 primary rendezvous per period.
// Generated by dt_to_rho.py; standard DT sign convention.
// =====

// Perko A 10_161: DT [4, 12, -16, 14, -18, 2, 8, -20, -10, -6] (n=10, 20 arcs)
new Gate, a1, a2, a3, a4, a5, a6, a7, a8, a9, a10,
a11, a12, a13, a14, a15, a16, a17, a18, a19, a20 in {

  contract Gate(oi, oo, ui, uo) = {
    new l in {
      for (s <- oi) { oo!(*s) | l!(Nil) }
      | for (s <- ui & _ <- l) { uo!(*s) | Gate!(*oi, *oo, *ui, *uo) }
    }
  }
  |
  Gate!(*a20, *a1, *a3, *a4) // crossing 1: over a20->a1, under a3->a4
  | Gate!(*a2, *a3, *a11, *a12) // crossing 2: over a2->a3, under a11->a12
}

```



```

| Gate!(*a15, *a16, *a4, *a5) // crossing 3: over a15->a16, under a4->a5
| Gate!(*a6, *a7, *a13, *a14) // crossing 4: over a6->a7, under a13->a14
| Gate!(*a17, *a18, *a8, *a9) // crossing 5: over a17->a18, under a8->a9
| Gate!(*a10, *a11, *a1, *a2) // crossing 6: over a10->a11, under a1->a2
| Gate!(*a12, *a13, *a7, *a8) // crossing 7: over a12->a13, under a7->a8
| Gate!(*a19, *a20, *a14, *a15) // crossing 8: over a19->a20, under a14->a15
| Gate!(*a9, *a10, *a16, *a17) // crossing 9: over a9->a10, under a16->a17
| Gate!(*a5, *a6, *a18, *a19) // crossing 10: over a5->a6, under a18->a19
|
// current: one token per arc; conserved, period 20
a1!(Nil) | a2!(Nil) | a3!(Nil) | a4!(Nil) | a5!(Nil) | a6!(Nil) | a7!(Nil) |
a8!(Nil) | a9!(Nil) | a10!(Nil) | a11!(Nil) | a12!(Nil) | a13!(Nil) | a14!(Nil) |
a15!(Nil) | a16!(Nil) | a17!(Nil) | a18!(Nil) | a19!(Nil) | a20!(Nil)
}

```

References

- [1] L. G. Meredith. *Knots as processes* (working notes, 2005).
- [2] F1R3FLY.io. *f1r3node-rust*: a pure-Rust F1R3FLY node with a rholang interpreter. <https://github.com/F1R3FLY-io/f1r3node-rust>.
- [3] Operational Semantics in Logical Form (OSLF): a family of algorithms auto-generating logics and type systems from graph-structured lambda theories. Adequacy is a property of the target logic specification rather than of the generator, and holds for those members whose specification supplies both a conjunction and a negation.
- [4] V. Turaev. *Operator invariants of tangles and R-matrices* (for the tangle category and its generators/relations).
- [5] K. A. Perko, Jr. *On the classification of knots*. Proc. Amer. Math. Soc. **45** (1974), 262–266.
- [6] C. N. Little. *Non-alternating $\pm$ knots*. Trans. Roy. Soc. Edinburgh **39** (1900), 771–778.
- [7] *Perko pair*. Wikipedia; the two reduced diagrams have different writhes, refuting Little’s writhe-invariance claim (a Tait conjecture).
- [8] L. H. Kauffman. *State models and the Jones polynomial*. Topology **26** (1987), 395–407.
- [9] W. B. R. Lickorish. *An Introduction to Knot Theory*. Graduate Texts in Mathematics **175**, Springer, 1997.